RPC y Análisis de Tráfico

APELLIDOS:	GRUPO TIT31-16	NOMBRE:

Si va a realizar la práctica en el Escritorio Virtual de la UPM consulte el ANEXO: Notas para la realización del laboratorio en Escritorio Virtual, que puede encontrar el final de este documento

PRESENTACIÓN

En esta práctica vamos a aprender a crear y utilizar un API RPC con Python. Crearemos distintos procedimientos utilizando la librería de Python xmlrpc. Con esta librería vamos a construir un servidor RPC con varios procedimientos y vamos a crear diferentes servidores para poder utilizar estos procedimientos.

A continuación vamos a analizar el tráfico generado por las diferentes llamadas RPC mediante el analizador de protocolos Wireshark. Construiremos diferentes generadores de tráfico y analizaremos entonces el tráfico que se genera en diferentes situaciones.

Ejercicio 1. Proyecto PyCharm y creación de un servidor

Empezamos por abrir PyCharm y crear un proyecto nuevo con el nombre RPC-Servidor.

Dentro de este proyecto creamos el fichero SimpleServer.py.
En este fichero vamos a escribir el siguiente código:

Importar el paquete xmlrpc.server

```
import xmlrpc.server
```

A continuación definimos las funciones que se van a implementar en este servidor: add, subtract, multiply y divide.

```
def add(x, y):
    return x + y

def subtract(x, y):
    return x - y

def multiply(x, y):
    return x * y

def divide(x, y):
    return x // y
```

Una vez definidos los procedimientos que se a ofrecer el servidor, procederemos a definir el servidor que se va a utilizar, para ello escribiremos a continuación las siguientes instrucciones para crear un servidor en localhost escuchando en el puerto 9000:

```
server = xmlrpc.server.SimpleXMLRPCServer(("localhost", 9000))
print("Listening on port 9000...")
```

Una vez definido el servidor, debemos añadirles las funciones que hemos definido con anterioridad, para ello escribiremos el siguiente código:

```
server.register_function(add, 'add')
server.register_function(subtract, 'subtract')
server.register_function(multiply, 'multiply')
server.register_function(divide, 'divide')
```

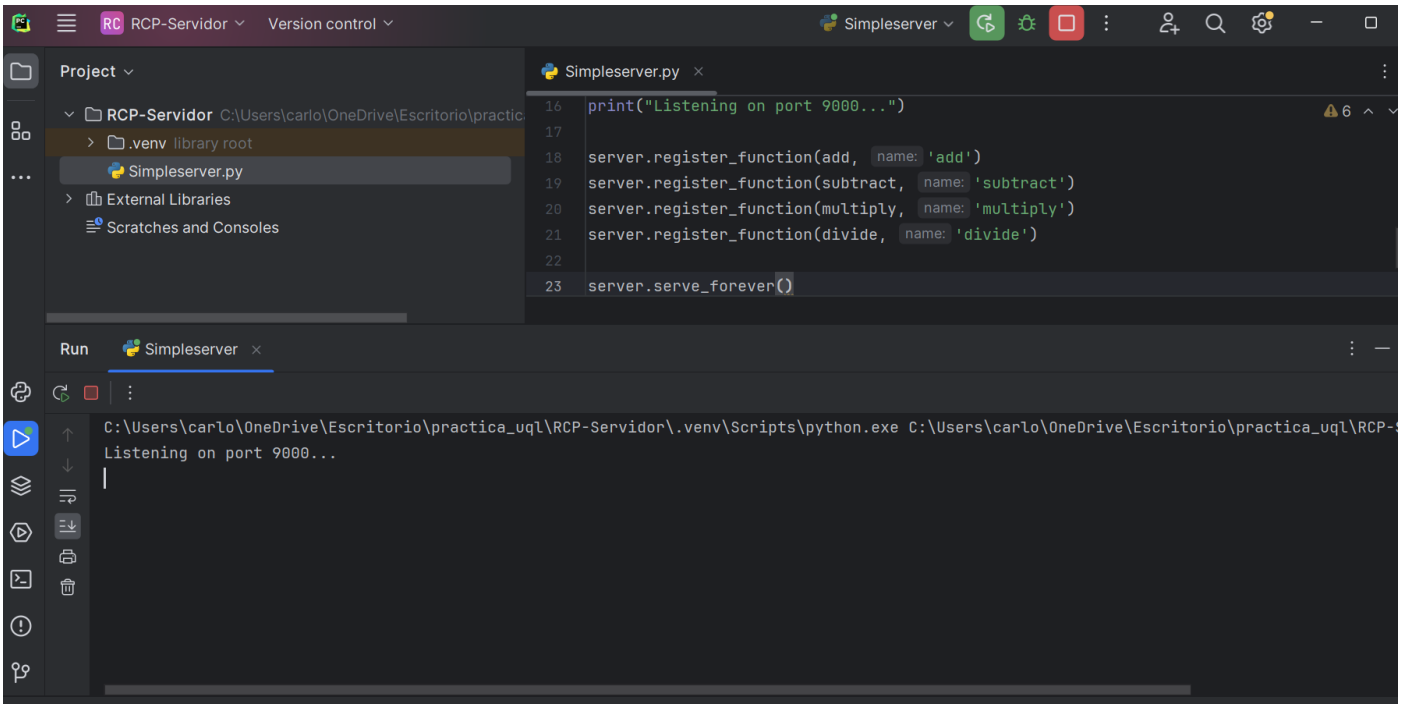
Mediante este código registramos cada uno de los procedimientos para que puedan ser utilizados de forma remota. En concretos hemos registrado los procedimientos add, subtract, multiply y divide con sus correspondientes nombre.

Para completar la definición del servidor, aun debemos añadir una instrucción más para ejecutar el servidor en bucle de forma indefinida:

```
server.serve_forever()
```

En este momento podemos ejecutar el servidor simplemente posicionando el cursos sobre el fichero SimpleServer.py y seleccionando Run 'simpleServer'

INSERTE UNA CAPTURA DE LA VENTANA DE EJECUCIÓN DE PYCHARM AL EJECUTAR SIMPLESERVER.PY:



Para detener la ejecución podemos pulsar en el cuadrado rojo que observamos a la izquierda de la ventana de ejecución.

Ejercicio 2. Crear un cliente RPC

Ahora vamos a crear un cliente para utilizar el servidor RPC que hemos definido antes. Para ello creamos el proyecto RPC-Client en **PyCharm** e indicamos que queremos abrirlo en una New Window". Dentro de este proyecto creamos el fichero el fichero SimpleClient.py. En este fichero en primer lugar importados el paquete xmlrpc.client

```
import xmlrpc.client
```

Creamos el objeto server local para conectado al servidore remoto que acabamos de definir en SimpleServer:

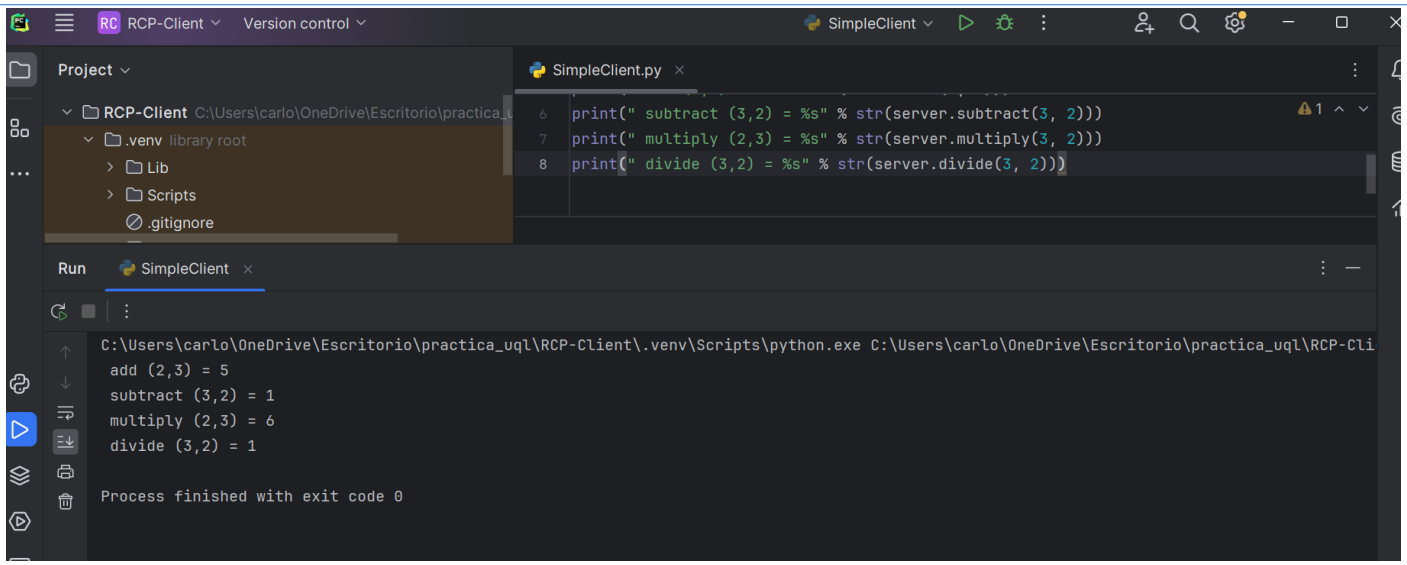
```
server = xmlrpc.client.ServerProxy('http://localhost:9000')
```

Como podemos ver nos estamos conectando al servicio web en localhost y en el puerto 9000. Tanto la url como el puerto al que nos conectamos deberá coincidir con los del servidor. A continuación podemos realizar llamadas a cualquiera de los métodos definidos en el servidor:

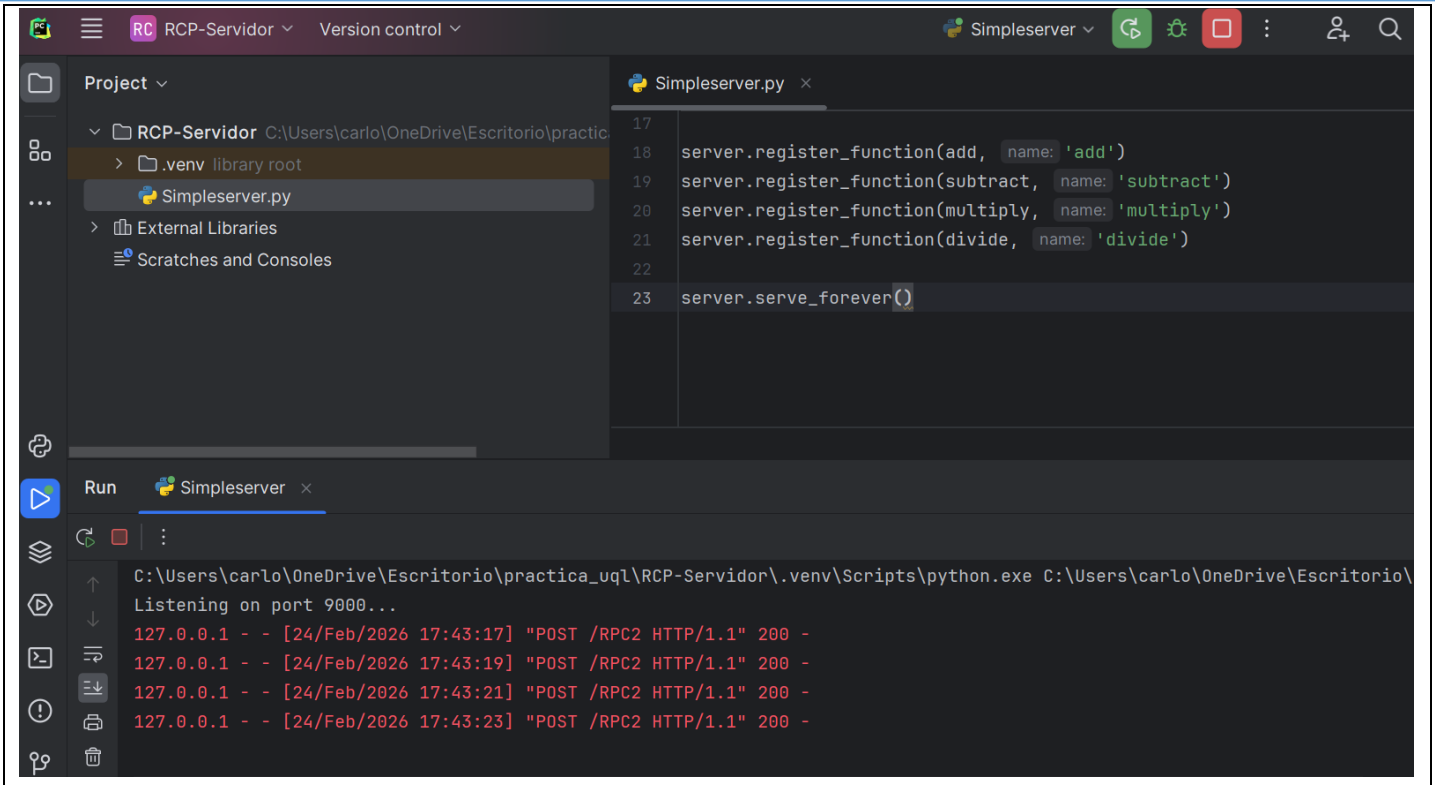
```
print (" add (2,3) = %s" % str(server.add(2,3)))
print (" subtract (3,2) = %s" % str(server.subtract(3,2)))
print (" multiply (2,3) = %s" % str(server.multiply(3,2)))
print (" divide (3,2) = %s" % str(server.divide(3,2)))
```

Ahora podemos ejecutar el servidor definido en SimpleServer y el cliente definido en SimpleClient que acabamos de definir.

ESCRIBA EL RESULTADO DE LA VENTANA DE RUN AL EJECUTAR SIMPLECLIENT.PY:



ESCRIBA EL RESULTADO DE LA VENTANA DE RUN AL EJECUTAR SIMPLESERVER.PY Y REALIZAR LAS LLAMADAS DESDE SIMPLECLIENT.PY:



Como puede observar en el servidor aparecen una serie de trazas que nos permiten ver las llamadas que se realizan al servidor RPC mediante http. Para desactivar las trazas debemos crear el servidor con la opción de logRequests a False:

```
server = xmlrpc.server.SimpleXMLRPCServer(("localhost", 9000), logRequests=False)
```

Puede probar a activar y desactivas el log, pero sugerimos mantener las trazas activas en el resto de esta práctica.

Ejercicio 3. Servidor RPC para análisis de tráfico

En este apartado vamos a crear un servidor RPC con dos métodos: el método ping que simplemente devuelve True cuando lo llamamos y el método echo que devuelve la misma información que le hemos enviado

Empezamos por crear dentro del proyecto RPC-Servidor el fichero EchoServer con el siguiente código:

```
from xmlrpc.server import SimpleXMLRPCServer

def echo(v):
    " Devuelve el mismo valor después de esperar un tiempo determinado"
    return v

def ping ():
    "devuelve TRUE"
    return "TRUE"

server = SimpleXMLRPCServer(("localhost", 9000))
#server.register_multicall_functions()
server.register_function(echo, "echo")
server.register_function(ping, "ping")

try:
    print('Use Control-C to exit')
    server.serve_forever()
except KeyboardInterrupt:
    print('Exiting')
```

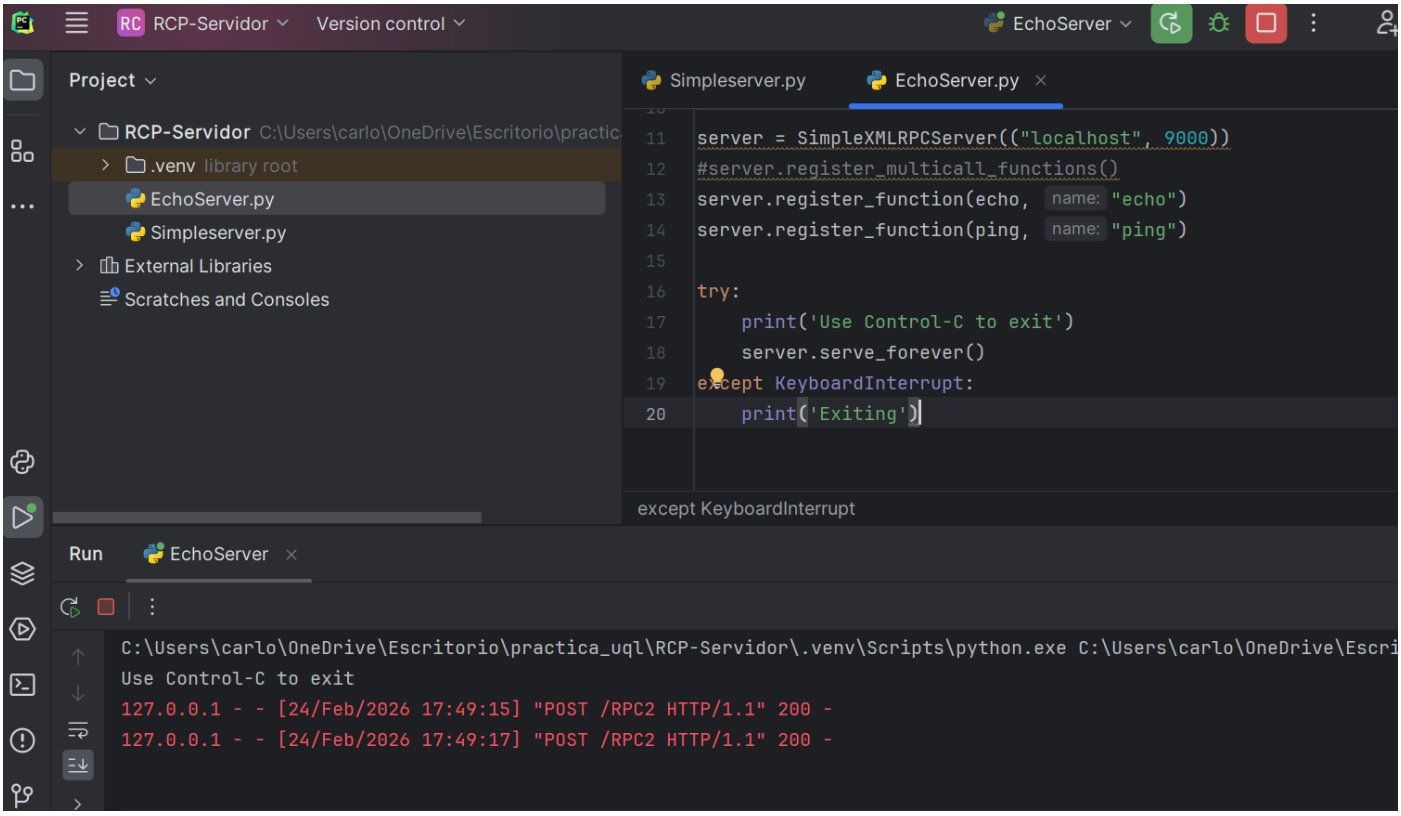
Creamos a continuación dentro del proyecto RPC-Cliente el fichero EchoClient con el siguiente código:

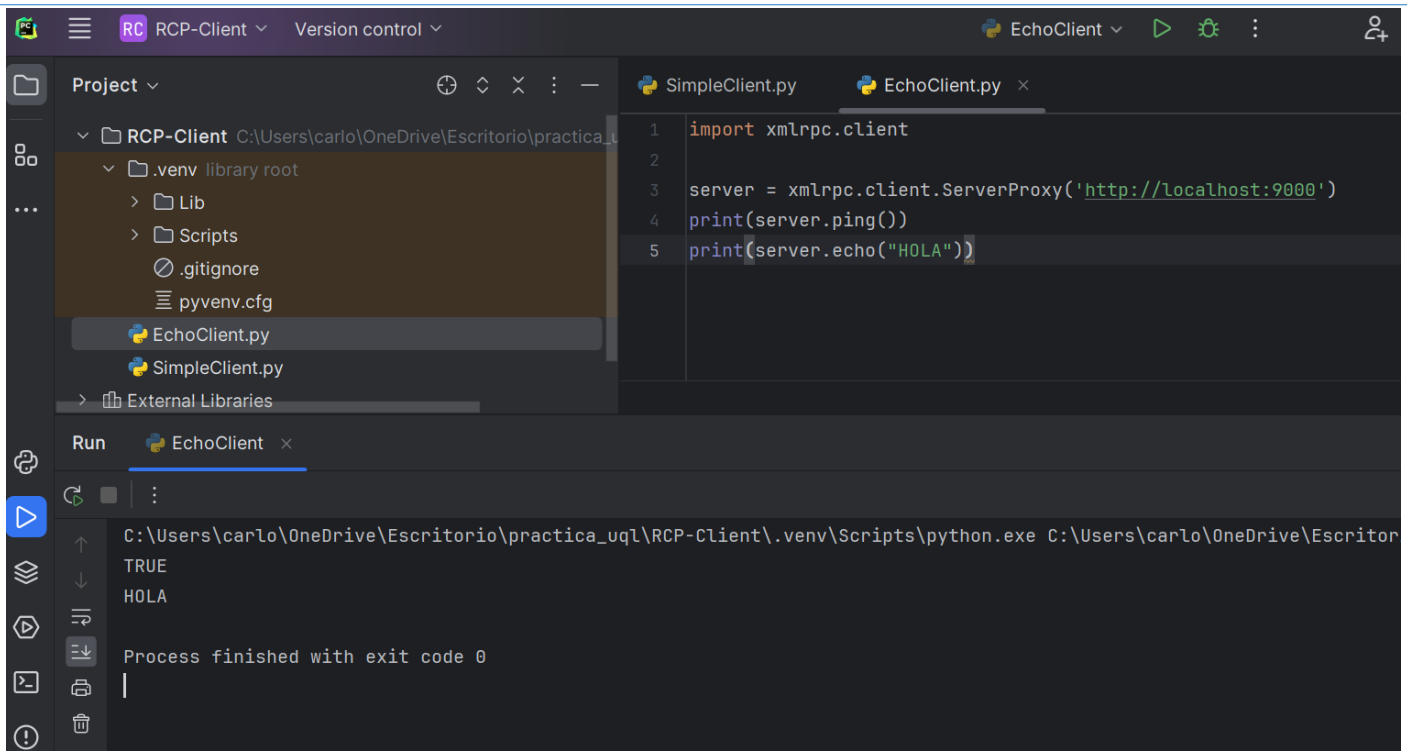
```
import xmlrpc.client

server = xmlrpc.client.ServerProxy('http://localhost:9000')
print (server.ping())
print (server.echo("HOLA"))
```

Podemos ejecutar EchoServer y EchoClient y observar el resultado.

ESCRIBA EL RESULTADO DE LA VENTANA DE RUN AL EJECUTAR ECHOSERVER.PY Y ECHOCLIENT.PY:




RCP-Client
Version control
EchoClient
Project
RCP-Client C:\Users\carlo\OneDrive\Escritorio\practica_u
.venv library root
Lib
Scripts
.gitignore
pyvenv.cfg
EchoClient.py
SimpleClient.py
External Libraries
SimpleClient.py
EchoClient.py
import xmlrpc.client
server = xmlrpc.client.ServerProxy('http://localhost:9000')
print(server.ping())
print(server.echo("HOLA"))
Run
EchoClient
C:\Users\carlo\OneDrive\Escritorio\practica_uql\RCP-Client\.venv\Scripts\python.exe C:\Users\carlo\OneDrive\Escritor
TRUE
HOLA
Process finished with exit code 0

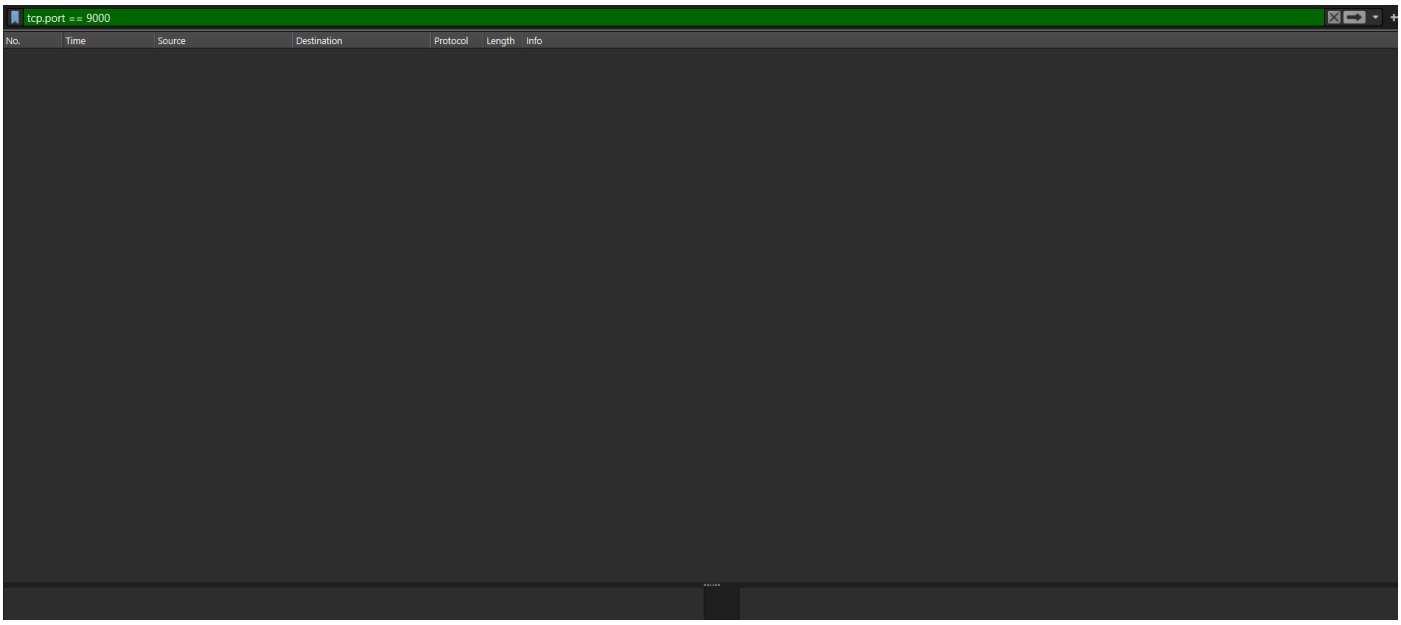
Ejercicio 4. Análisis del tráfico de las llamadas de EchoClient a EchoServer

Ahora vamos a analizar el tráfico entre cliente y servidor generado por RPC. Para ello vamos a utilizar el analizador de protocolos Wireshark.

Ejecutamos en primer lugar Wireshar y seleccionamos el interfaz Loopback: lo0 para iniciar la captura. Esto nos permite capturar todo el tráfico intercambiado dentro de nuestro ordenador.

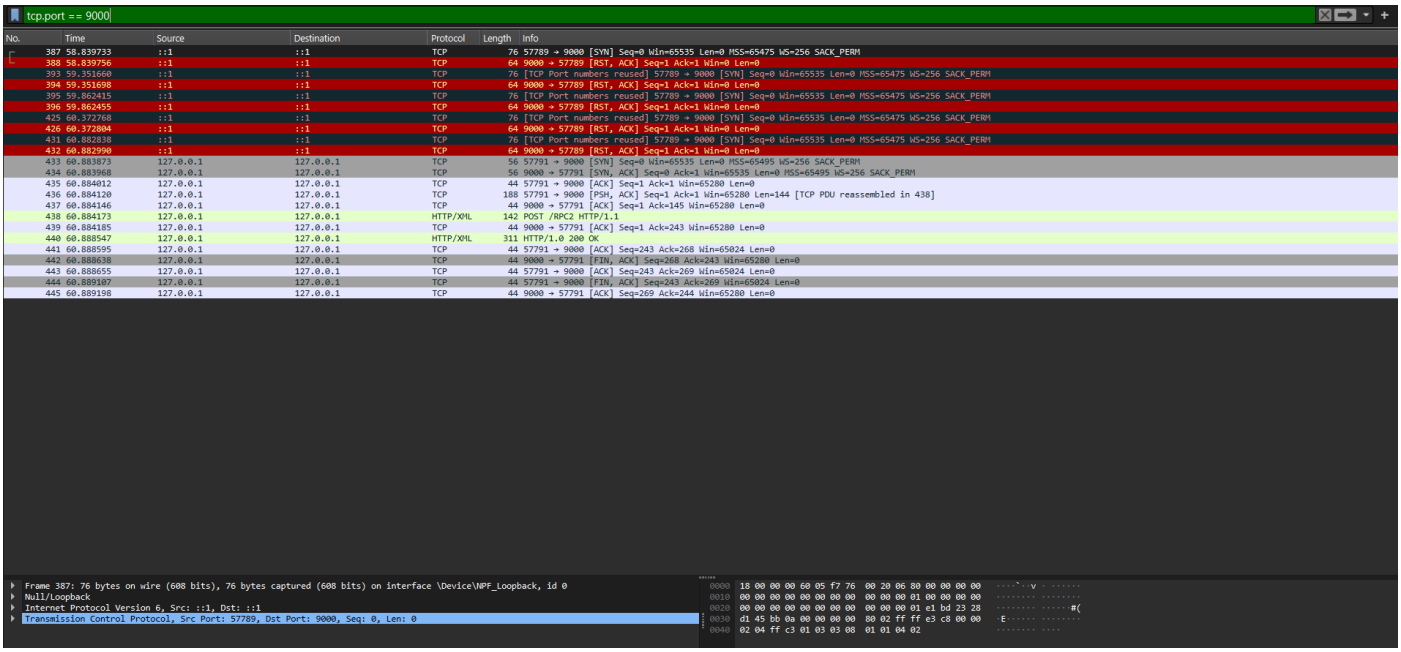
A continuación iniciamos el servidor EchoServer desde PyCharm. En este momento no debemos observar ningún tráfico relevante en Wireshar. Ahora vamos a ejecutar EchoClient, pero sólo la parte de creación del server, sin realizar las llamadas a los procedimientos ping ni echo. Esto lo podemos realizar simplemente comentado las líneas de llamada o utilizando el depurador de PyCharm.

INSERTE UNA CAPTURA DE LA VENTANA DE CAPURA DE WIRESHARK AL TERMINAR LA EJECUCIÓN DE ECHOCLIENT SIN REALIZAR NINGUNA LLAMADA A PROCEDIMIENTO



Ahora vamos a repetir el mismo procedimiento del caos anterior, pero ejecutando la llamada al procedimiento ping.

INSERTE UNA CAPTURA DE LA VENTANA DE CAPURA DE WIRESHARK AL TERMINAR LA EJECUCIÓN DE ECHOCLIENT REALIZANDO LLA LLAMADA AL PROCEDIMIENTO PING

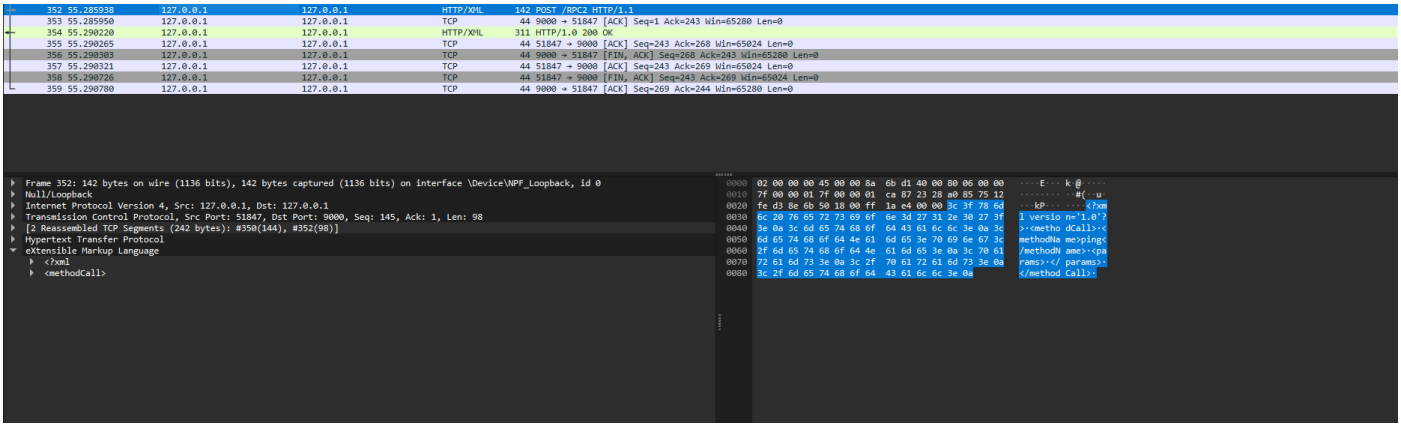


Indique a continuación:

Cuantos paquetes se han intercambiado: _____23_____

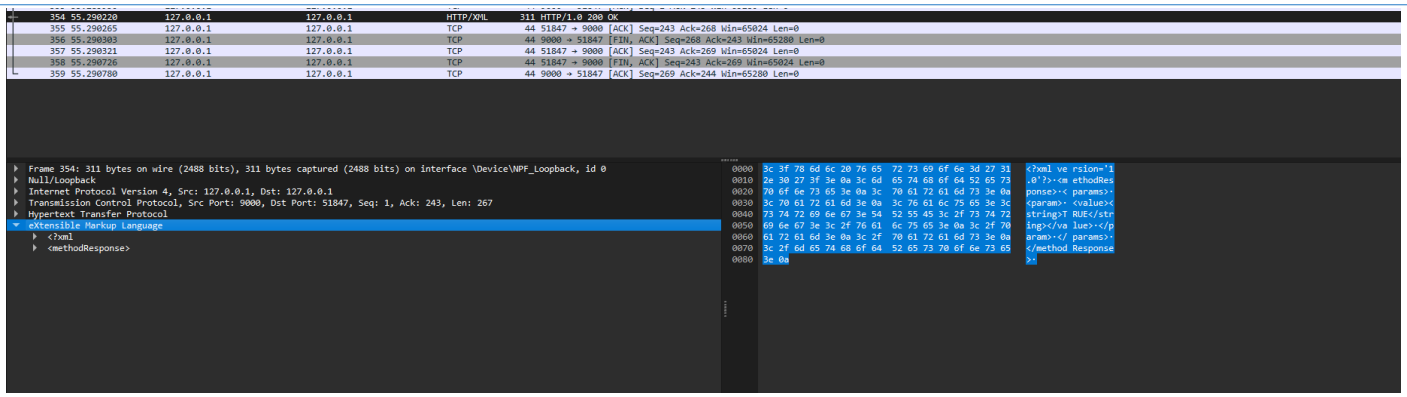
Tamaño total de los paquetes intercambiado: _____1761 bytes_____

IDENTIFIQUE Y CAPTURE EL XML QUE ENVÍA EL CLIENTE AL SERVIDOR PARA REALIZAR LA LLAMADA AL PROCEDIMIENTO PING



Tamaño en bytes de la petición XML: _____98 bytes_____

IDENTIFIQUE Y CAPTURE EL XML QUE ENVÍA EL SERVIDOR AL CLIENTE COMO RESULTADO DE LA LLAMADA AL PROCEDIMIENTO PING:



Tamaño en bytes de la respuesta en XML: ____152 bytes_____

Observando las trazas calcule el tiempo total empleado en el intercambio de mensajes para completar la llamada a PING:

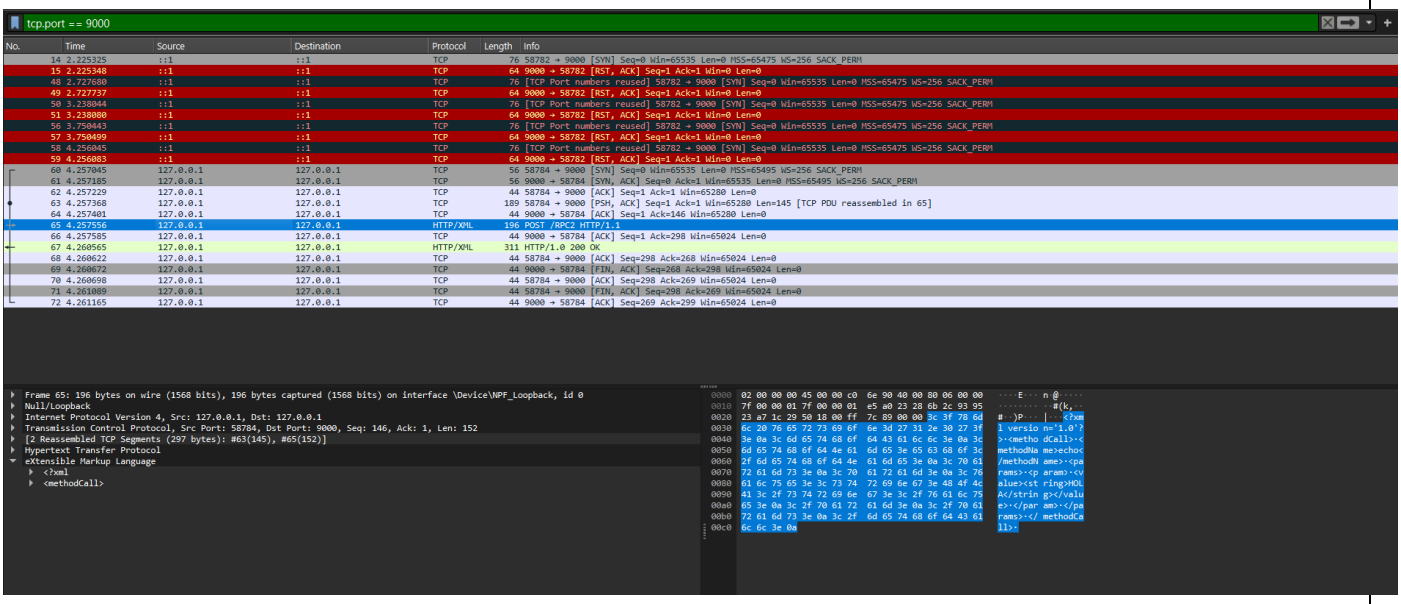
INDIQUE EL TIEMPO Y COMO OBTENERLO A CONTINUACIÓN

Usamos el timestamp del primer paquete como referencia: 52.259902
Y el timestamp del ultimo paquete: 55.290780

Tiempo total empleado para el completar el intercambio de mensajes de ping: 55.290780 - 52.259902 = 3.030878 segundos

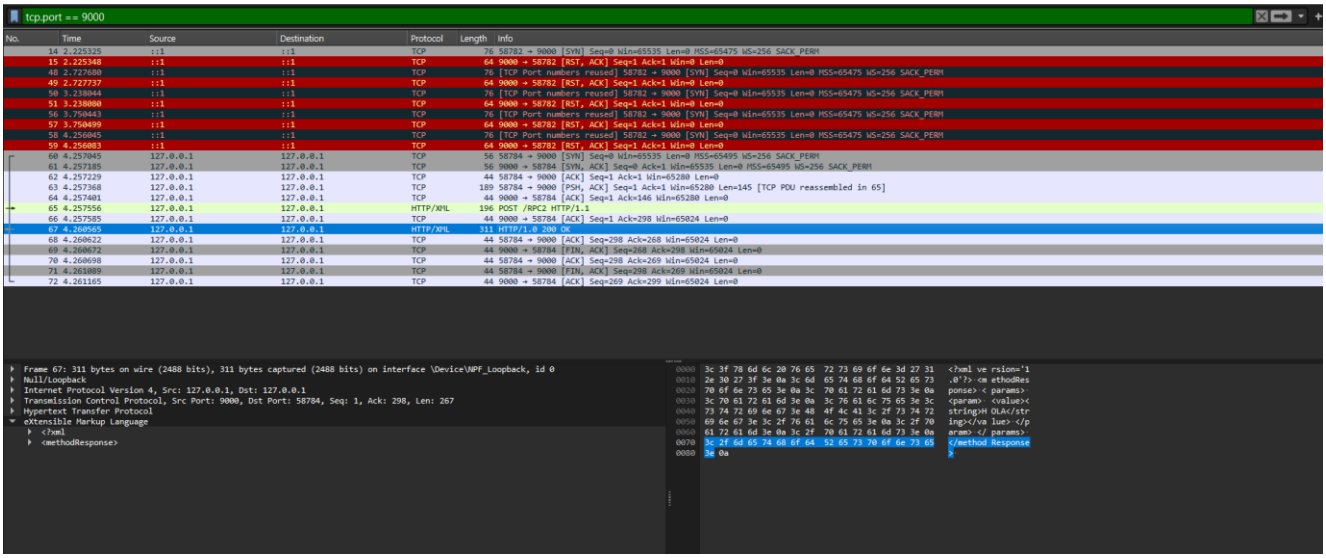
Realice el mismos proceso para la llamada al método echo

IDENTIFIQUE Y CAPTURE EL XML QUE ENVÍA EL CLIENTE AL SERVIDOR PARA REALIZAR LA LLAMADA AL PROCEDIMIENTO ECHO



Tamaño en bytes de la petición XML: 152 bytes

IDENTIFIQUE Y CAPTURE EL XML QUE ENVÍA EL SERVIDOR AL CLIENTE COMO RESULTADO DE LA LLAMADA AL PROCEDIMIENTO ECHO:



Tamaño en bytes de la respuesta en XML: 130 bytes

Todo es similar al caso anterior, pero ahora el XML de la llamada es 152 bytes y el XML de la respuesta es 130 bytes

Observando las trazas calcule el tiempo total empleado en el intercambio de mensajes para completar la llamada a ECHO:

INDIQUE EL TIEMPO Y COMO OBTENERLO A CONTINUACIÓN

Tomando en cuenta los handshake
Usamos el timestamp del primer paquete como referencia: 2.225325
Y el timestamp del ultimo paquete: 4.261165

Tiempo total empleado para el completar el intercambio de mensajes de ping: 4.261165 - 2.225325 = 2.03584 segundos

Finalmente podemos ejecutar el cliente con las dos llamadas a procedimiento originales ping y echo seguidas.

INDIQUE SI OBSERVA ALGUNA DIFERENCIA ENTRE LOS CASOS DE LLAMADAS A UN SOLO PROCEDIMIENTO Y OS DE LLAMADAS CONSECUTIVAS DESDE EL MISMO CLIENTE.

Llamadas consecutivas desde el mismo cliente se realizan independientemente y de manera asíncrona, es decir, la llamada de ping y el echo no se esperan una a la otra, de modo que no esperan a que termine la que se ha llamado primero

Calcule el tiempo total empleado en el intercambio de mensajes para completar las llamadas a PING y ECHO.

INDIQUE EL TIEMPO Y COMO OBTENERLO A CONTINUACIÓN

Tomando en cuenta los handshake
Usamos el timestamp del primer paquete como referencia: 2.320580
Y el timestamp del ultimo paquete: 6.406515

Tiempo total empleado para el completar el intercambio de mensajes de ping: 6.406515 - 2.320580 = 4.085935 segundos

Ejercicio 5. Análisis del tráfico de las llamadas a EchoServer de forma concurrente

También podemos plantearnos realizar llamadas concurrentes a los procedimientos ping y echo de forma concurrente, no consecutiva.

Como hemos visto en el apartado anterior los tiempos de respuesta son muy pequeños, del orden de los pocos milisegundos, por lo que para poder realizar un análisis de posibles llamada simultáneas, vamos a necesitar aumentar el tiempo que tarda en responder nuestro servidor.

Creamos un nuevo fichero EchoServerDelay.py donde ahora los procedimientos echo y ping esperan un cierto tiempo (en segundos) antes de responder, como se indica a continuación:

```
from xmlrpc.server import SimpleXMLRPCServer
import time
import sys


def echo(v):
    " Devuelve el mismo valor después de esperar un tiempo determinado"
    time.sleep(t)
    return v

def ping ():
    "Devuelve TRUE después de esperar un tiempo determinado"
    time.sleep(t)
    return "TRUE"


 ……
```

Definiremos t como una variable que iniciamos en el main del programa para poder configurar de forma más sencilla este parámetro si fuera necesario

Para poder ejecutar los dos clientes de forma simultánea, lo más práctico es crear dos proyectos: RPC-Client-Echo y RPC-Client-Ping. En el fichero main.py de cada uno de los proyectos podemos crear el cliente echo y el cliente ping como se ha indicado anteriormente.

Reiniciamos la captura de WireShark, ejecutamos el servidor con 10 segundos de delay y procedemos a ejecutar los clientes de echo y ping para que las dos llamadas coincidan en el servidor.

Analizando las tracas de WiresShark calcule:

Tiempo entre la recepción de la llamada a ping y a echo: _____
Tiempo entre la respuesta a ping y a echo: _____

ESCRIBAS LAS OBSERVACIONES SOBRE EL RESULTADO QUE SE OBTIENE CON WIRESHARK :

Se toma el timestamp del primer paquete POST (`347.593732100`) y el del segundo paquete POST (`350.239429200`) en Wireshark.
Se calcula la diferencia: `350.239429200 - 347.593732100 = 2.645697100 s`.
Tiempo entre la recepción de la llamada a ping y a echo: 2.645697100 s (≈ 2.646 s).

154	347.593732100	127.0.0.1	127.0.0.1	HTTP/X…	196 POST /RPC2 HTTP/1.1
155	347.593771200	127.0.0.1	127.0.0.1	TCP	44 9000 → 53648 [ACK] Seq=1 Ack=298 Win=65024 Len=0
156	348.185311500	::1	::1	TCP	76 53649 → 9000 [SYN] Seq=0 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
157	348.185372400	::1	::1	TCP	64 9000 → 53649 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
158	348.698714300	::1	::1	TCP	76 [TCP Port numbers reused] 53649 → 9000 [SYN] Seq=0 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
159	348.698779200	::1	::1	TCP	64 9000 → 53649 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
160	349.209260100	::1	::1	TCP	76 [TCP Port numbers reused] 53649 → 9000 [SYN] Seq=0 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
161	349.209348700	::1	::1	TCP	64 9000 → 53649 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
162	349.723989200	::1	::1	TCP	76 [TCP Port numbers reused] 53649 → 9000 [SYN] Seq=0 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
163	349.724049700	::1	::1	TCP	64 9000 → 53649 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
164	350.237948000	::1	::1	TCP	76 [TCP Port numbers reused] 53649 → 9000 [SYN] Seq=0 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
165	350.237990200	::1	::1	TCP	64 9000 → 53649 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
166	350.238877600	127.0.0.1	127.0.0.1	TCP	56 53650 → 9000 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
167	350.239061000	127.0.0.1	127.0.0.1	TCP	56 9000 → 53650 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
168	350.239136900	127.0.0.1	127.0.0.1	TCP	44 53650 → 9000 [ACK] Seq=1 Ack=1 Win=65280 Len=0
169	350.239355400	127.0.0.1	127.0.0.1	TCP	188 53650 → 9000 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=144 [TCP PDU reassembled in 171]
170	350.239387800	127.0.0.1	127.0.0.1	TCP	44 9000 → 53650 [ACK] Seq=1 Ack=145 Win=65280 Len=0
171	350.239429200	127.0.0.1	127.0.0.1	HTTP/X…	142 POST /RPC2 HTTP/1.1

La diferencia temporal entre las respuestas HTTP `200` OK es de **10.0028177 s**, calculada a partir de los timestamps de Wireshark (líneas 173 y 179). Este valor coincide aproximadamente con el retardo configurado (`t = 10 s`) y evidencia que el servidor atiende las peticiones de forma secuencial.

173	357.595854800	127.0.0.1	127.0.0.1	HTTP/X…	311 HTTP/1.0 200 OK
174	357.596269600	127.0.0.1	127.0.0.1	TCP	44 53648 → 9000 [ACK] Seq=298 Ack=268 Win=65024 Len=0
175	357.596765200	127.0.0.1	127.0.0.1	TCP	44 9000 → 53648 [FIN, ACK] Seq=268 Ack=298 Win=65024 Len=0
176	357.596852600	127.0.0.1	127.0.0.1	TCP	44 53648 → 9000 [ACK] Seq=298 Ack=269 Win=65024 Len=0
177	357.596895900	127.0.0.1	127.0.0.1	TCP	44 53648 → 9000 [FIN, ACK] Seq=298 Ack=269 Win=65024 Len=0
178	357.597067100	127.0.0.1	127.0.0.1	TCP	44 9000 → 53648 [ACK] Seq=269 Ack=299 Win=65024 Len=0
179	367.598672500	127.0.0.1	127.0.0.1	HTTP/X…	311 HTTP/1.0 200 OK

ANEXO: Notas para la realización del laboratorio en Escritorio Virtual

- Este Laboratorio debe realizar en el Escritorio Virtual: **ETSISI (Windows 11).**
- Recuerde que todos los ficheros, datos y otras informaciones que se generen durante la realización del Laboratorio se borraran al terminar la ejecución, **por lo que si los puede necesitar con posterioridad deberá descargarlos fuera del Escritorio Virtual.**
- Sobre PyCharm:
  - Para Ejecutar PyCharm pulse con el botón derecho del ratón sobre y escriba PyCharm en el buscador
  - Los proyectos se crean en: C:/Users/UPM/PyCharmProjects.
- Para subir y descargar ficheros al escritorio Virtual podemos utilizar el disco “UDSs en Guacamole RDP” que podemos ver con el navegador del escritorio virtual dentro de “Este equipo”.